Day 5 Assignment — Technical Writing Lab

Student Name: Zakariye Ali Mursal
Date: 12/10/2026
Course/Class: Software Engineering — PLP 2026

Exercise A — User Manual Procedure

Task documented: Setting up a GitHub repository from scratch and making a first commit

Target audience: A first-semester computing student with basic computer literacy and no prior experience with Git or GitHub.

PREREQUISITES

Before you begin, make sure you have all of the following:

1. A computer running Windows, macOS, or Linux with a terminal application available (Command Prompt or PowerShell on Windows; Terminal on macOS or Linux).

2. Git installed. Open a terminal and type: git --version
If you see a version number (for example, "git version 2.43.0"), Git is installed. If you see an error, install Git from https://git-scm.com before continuing.

3. A free GitHub account. If you do not have one, sign up at https://github.com.

4. A personal access token. GitHub no longer accepts account passwords for Git operations. A personal access token is a string that acts as a password, but is scoped to specific permissions and can be revoked at any time. Generate one at https://github.com/settings/tokens — choose "Generate new token (classic)", set the expiration to 90 days, check the "repo" scope, and copy the token to a safe place. You will paste it when Git asks for a password.

5. A project folder on your computer with at least one file you want to track with Git. For this procedure, we will assume the folder is named "my-first-project".

STEP-BY-STEP PROCEDURE

Step 1. Open your terminal application.
Expected result: A command prompt appears, waiting for input. On macOS or Linux it ends with $; on Windows it ends with >.

Step 2. Navigate into your project folder by typing cd path/to/my-first-project (replace with the actual path) and pressing Enter.
Expected result: The prompt changes to show you are inside the project folder. Type "ls" on macOS or Linux, or "dir" on Windows, to confirm — your project files appear in the list.

Step 3. Initialize a Git repository by typing git init and pressing Enter.
Expected result: A message ending with "Initialized empty Git repository" appears. A hidden .git folder is now inside your project.

Step 4. Stage all files by typing git add . and pressing Enter.
Expected result: No message appears. Silence is normal — git add does not print output on success.

Step 5. Verify what is staged by typing git status and pressing Enter.
Expected result: Your files appear in green under "Changes to be committed". If any appear in red, they are not staged — run "git add ." again.

Step 6. Create your first commit by typing git commit -m "Initial commit" and pressing Enter.
Expected result: A message ending with a line like "1 file changed, 3 insertions(+)" appears. Your commit is now saved locally.

Step 7. In your browser, go to https://github.com and sign in. Click the "+" icon in the top-right corner, then click "New repository".
Expected result: The "Create a new repository" page opens.

Step 8. In the "Repository name" field, type "my-first-project". Leave all other options at their defaults. Do NOT check "Add a README", "Add .gitignore", or "Choose a license". You should leave these unchecked because your local project already contains files, and GitHub adding its own files would create a conflict the first time you push. Click "Create repository".
Expected result: A "Quick setup" page loads with commands you will use in the next steps.

Step 9. In your terminal, connect your local repository to GitHub by typing git remote add origin https://github.com/YOUR-USERNAME/my-first-project.git (replace YOUR-USERNAME with your GitHub username) and pressing Enter.
Expected result: No message appears. Type "git remote -v" and press Enter — you should see the URL listed twice.

Step 10. Rename your branch to "main" by typing git branch -M main and pressing Enter.
Expected result: No message appears. Type "git branch" and press Enter — you should see "* main", confirming you are on the main branch.

Step 11. Push your commit to GitHub by typing git push -u origin main and pressing Enter.
Expected result: You are prompted for a username and password. Type your GitHub username, then paste your personal access token when asked for the password (the terminal will not show any characters as you paste — this is normal). On success, you will see a line ending with "* [new branch] main -> main".

Step 12. Open your browser and go to https://github.com/YOUR-USERNAME/my-first-project.
Expected result: Your repository page loads, showing your project files and your commit message. The push succeeded.

SCREENSHOT DESCRIPTION

What to capture: A screenshot of the terminal immediately after Step 11 completes.

What the screenshot should show:
- The terminal window displaying the successful push output, including the line "* [new branch] main -> main".
- The command prompt below the output, confirming the command completed.
- If visible, the folder path in the prompt showing you were in the correct project directory.

Why this screenshot matters: It proves the entire flow worked — the local commit was created and successfully pushed to GitHub — in a single image. It is the most useful piece of visual evidence for this procedure.

TROUBLESHOOTING — TWO COMMON BEGINNER ERRORS

Error 1: Authentication failed

Symptom: You run "git push -u origin main" and see:
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/YOUR-USERNAME/...'

Cause: GitHub stopped accepting account passwords for Git operations in 2021. If you typed your GitHub account password at the password prompt, authentication fails.

Fix:
1. Go to https://github.com/settings/tokens.
2. Click "Generate new token" → "Generate new token (classic)".
3. Give it a name (for example, "my-first-project"), set expiration to 90 days, and check the top-level "repo" scope.
4. Click "Generate token" and copy the value (it starts with "ghp_"). It is shown only once.
5. Back in the terminal, run "git push -u origin main" again.
6. At the Username prompt, type your GitHub username.
7. At the Password prompt, paste the token (the terminal shows nothing as you paste — this is normal).
8. Press Enter. The push should now succeed.

Prevention: Save your token somewhere safe. You will use it every time you push to a new repository.

Error 2: "remote origin already exists"

Symptom: You run "git remote add origin https://github.com/YOUR-USERNAME/my-first-project.git" and see:
error: remote origin already exists.

Cause: You have already added a remote named "origin" to this repository. This often happens if you re-ran the command or copied it twice.

Fix: Use "git remote set-url" instead of "git remote add" to update the existing remote:
  git remote set-url origin https://github.com/YOUR-USERNAME/my-first-project.git

Verify with "git remote -v" — you should see the correct URL listed twice. Then continue with the push.

Exercise B — API Reference Entry

Create Task

Creates a new task in a specified project. The task is assigned to a user, given a due date, and marked with a priority level. This endpoint can only be called by an authenticated user who is a member of the project.

ENDPOINT

POST /api/v1/projects/{project_id}/tasks

AUTHENTICATION

All requests must include a valid API key in the Authorization header.

Header: Authorization
Value: Bearer YOUR_API_KEY
Required: Yes

Header: Content-Type
Value: application/json
Required: Yes

Header: Accept
Value: application/json
Required: Yes

PATH PARAMETERS

project_id — string (UUID) — Required — The unique identifier of the project the task belongs to.

REQUEST BODY

title — string — Required — The task title. Maximum 200 characters.

description — string — Optional — A detailed description of the task. Maximum 2000 characters.

assignee_id — string (UUID) — Required — The unique identifier of the user assigned to the task. This field is required because every task in this system must have an owner — a design decision that ensures accountability.

due_date — string (ISO 8601 date) — Required — The date the task is due, in YYYY-MM-DD format.

priority — string (enum) — Required — The priority level. Must be one of: "low", "medium", or "high".

EXAMPLE REQUEST

HTTP headers:
POST /api/v1/projects/3f7a9c1e-1d4b-4b9c-9c5a-7e2f9a8b3c21/tasks
Authorization: Bearer sk_live_abc123def456ghi789
Content-Type: application/json
Accept: application/json

JSON body:
{
  "title": "Prepare Q4 sales report",
  "description": "Compile sales figures from all regional offices and prepare a summary report for the leadership team.",
  "assignee_id": "8a4d2e1f-9b3c-4a5d-8f6e-1c2b3a4d5e6f",
  "due_date": "2026-11-15",
  "priority": "high"
}

RESPONSE CODES

201 Created — The task was successfully created. The response body contains the new task's details.

400 Bad Request — The request body is malformed, missing a required field, or contains an invalid value (for example, priority is not low/medium/high, or due_date is not in YYYY-MM-DD format).

401 Unauthorized — The Authorization header is missing, malformed, or the API key is invalid or expired.

403 Forbidden — The authenticated user is not a member of the specified project and does not have permission to create tasks in it.

404 Not Found — No project exists with the given project_id, or the assignee_id does not match any user.

409 Conflict — A task with the same title already exists in the project and duplicates are not allowed.

422 Unprocessable Entity — The request is syntactically valid but semantically incorrect — for example, due_date is in the past, or title exceeds 200 characters.

429 Too Many Requests — The client has exceeded the rate limit. Retry after the time specified in the Retry-After header.

500 Internal Server Error — Something went wrong on our side. Retry the request; if the problem persists, contact support.

RATE LIMITS

This endpoint is limited to 100 requests per minute per API key. When the limit is exceeded, the API returns 429 with a Retry-After header specifying how many seconds to wait before retrying. Clients should implement exponential backoff when handling 429 responses to avoid retry storms.

EXAMPLE SUCCESSFUL RESPONSE (201 Created)


```
{
  "id": "c1b2a3d4-e5f6-4789-9abc-def012345678",
  "project_id": "3f7a9c1e-1d4b-4b9c-9c5a-7e2f9a8b3c21",
  "title": "Prepare Q4 sales report",
  "description": "Compile sales figures from all regional offices and prepare a summary report for the leadership team.",
  "assignee": {
    "id": "8a4d2e1f-9b3c-4a5d-8f6e-1c2b3a4d5e6f",
    "name": "Amina Yusuf",
    "email": "amina.yusuf@example.com"
  },
  "due_date": "2026-11-15",
  "priority": "high",
  "status": "open",
  "created_at": "2026-10-12T09:34:11Z",
  "updated_at": "2026-10-12T09:34:11Z",
  "created_by": {
    "id": "d2c3b4a5-e6f7-4890-8bcd-efa123456789",
    "name": "Zakariye Ali Mursal"
  }
}
```


EXAMPLE ERROR RESPONSE (400 Bad Request)


```
{
  "error": {
    "code": "invalid_request",
    "message": "The field 'priority' must be one of: low, medium, high.",
    "field": "priority"
```

```
  }
}
```

Peer Critique

Purpose

The critique process is as important as the writing itself. It develops the professional skill of giving and receiving specific, constructive feedback — a skill that transfers well beyond technical writing.

Peer Critique — Exercise A (User Manual Procedure)

What worked well
1. The prerequisites section lists every tool needed, so a reader can verify their setup before starting.
2. Each step contains a single action and states the expected result, which makes the procedure easy to follow for a beginner.

What could be improved
1. Step 8 tells the reader not to check certain options on GitHub, but does not explain why. A short reason would help the reader understand the consequence of checking them.
2. The troubleshooting section covers the token error but not the "remote origin already exists" error, which is another common beginner mistake.

One question for the writer
Would a first-semester student know what a "personal access token" is from the prerequisites alone, or should there be a one-sentence explanation of what it is and why GitHub requires it?

Peer Critique — Exercise B (API Reference Entry)

What worked well
1. Every request field lists its type, whether it is required, and a description — exactly what a developer needs to form a valid request.
2. The response codes list explains when each code occurs in plain language, not just what the code name means.

What could be improved
1. The example request mixes HTTP headers and JSON body without a clear visual separation. Splitting them into two labeled blocks would be easier to scan.
2. There is no rate limit section, even though 429 is documented. A short note on the actual limit and how to handle it would complete the entry.

One question for the writer

Is "assignee_id" required, or could a task be created unassigned and assigned later? The current documentation treats it as required — a design decision worth documenting explicitly.

Reflection on receiving feedback

After the critique, the writer should:
1. Thank the reviewer for specific feedback rather than general praise or criticism.
2. Identify which suggestions to act on and which to defer, with a short reason for each.
3. Update the document where appropriate.
4. Note what they learned about their own writing from the critique — patterns in what reviewers noticed, gaps they didn't see, or assumptions they made about the reader.

Reflection

The reviewer pointed out that Step 8 tells readers not to check certain GitHub options without explaining why. On reflection, this is a real gap — a beginner might check "Add a README" because it sounds helpful, then encounter a merge conflict on the first push. I have added a one-sentence explanation of the reason. The reviewer also noted that the troubleshooting section covers only one error; I have added the "remote origin already exists" case, since it is a common beginner mistake. The reviewer also flagged that the prerequisites mentioned "personal access token" without explaining what it is; I have added a one-line definition. Finally, the reviewer noted that the API reference had no rate limit section even though 429 was documented — I have added a dedicated RATE LIMITS section, and I split the example request into labeled HTTP headers and JSON body blocks for clarity. What I learned from this critique is that I tend to assume the reader understands the "why" behind my instructions — adding short reasons and definitions will make the documentation more resilient for a true beginner.

END OF DOCUMENT